

Skip – List
–Search Element
–Program Analysis

Program Analysis:

```
void searchElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;

    // Traverse from highest level to the lowest
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
    }

    // Step into the exact match at level 0
    current = current->forward[0];

    if (current != NULL && current->key == key) {
        cout << "Found key " << key << " in the Skip List." << endl;
    } else {
        cout << "Key " << key << " not found." << endl;
    }
}

...
case 3:
    cout << "Enter key to search: ";
    cin >> key;
    searchElement(list, key);
    break;
```

Time Complexity, Space Complexity and Algorithm

A)Time Complexity

Note: –This searching part of the program, already discussed during insertion of element lets make it short:

1. Best Case Time Complexity: $O(1)$

The absolute best – case scenario occurs when the search halts immediately without horizontally traversing any nodes.

- ***The Condition:** This happens if the **list is completely empty**, or **if the target key is smaller than the very first element in the list.***
- ***The Execution:** The horizontal **while** loop condition **$(current \rightarrow forward[i] \rightarrow key < key)$** evaluates to **False** on the very first check at every single level.*
- ***The Result:** The algorithm completely ducks the horizontal traversal. The outer for loop simply drops down vertically, which takes a tiny, constant amount of time because the maximum level is a hard fixed constant.*

2. Worst Case Time Complexity: $O(n)$

The worst – case scenario occurs when the probabilistic balancing of the Skip List suffers a complete structural failure. This degrades the structure into a standard singly linked list, forcing a linear scan.

We can summarize this using either of the two extreme scenarios:

- ***Scenario A (The All Tails Flat List):*** *Every node generated a level of 0, meaning no express lanes exist. The algorithm is forced to traverse every single node one by one on Level 0.*
- ***Scenario B (The All Heads Traffic Jam):*** *Every node generated the maximum possible level (e. g. , Level 3). Because every node exists on the top floor, the pointer cannot skip over anything and is forced to perform a linear $O(n)$ traversal across the crowded top level.*

This we have discussed in Insert Element.

3. Average Case Time Complexity: $O(\log n)$

To summarize the average case without rewriting all the math, we only need to reference William Pugh's "Backward Walk" analysis.

- **The Expected Steps:** Because every node's height is determined by an independent coin flip with probability p , a geometric distribution dictates that the expected number of horizontal steps taken on any level before moving up is $\frac{1}{p}$.
- **The Expected Height:** The expected maximum height of the list, $L(n)$, where we expect to find exactly 1 node, is $\log_{\frac{1}{p}}(n)$.
- **The Total Cost:** The total expected search cost is the number of levels multiplied by the expected steps per level.

$$E[\text{search cost}] = \frac{L(n)}{p} + \frac{p}{1-p}$$

[This we have found out from William Pugh's Explanation and its adaptation to our – 0 based index program.]

- **The Conclusion:** When evaluated for a fair coin toss ($p = 0.5$), the constants drop away, leaving a strict average time complexity of $O(\log_2 n)$.

$O(\log_2 n)$ falls to : $O(\log n)$ category as $\frac{1}{p}$ and $\frac{p}{1-p}$ are constants

and $L(n) = \log_{\frac{1}{p}}(n)$.

B) Space Complexity

1. Input Space Complexity

The input space for the Search module is completely identical to the Insertion module because both functions act on the exact same data structure using the exact same arguments.

A. Parametric Input Space: $O(1)$

- *The search function takes the same parameters: a memory pointer addressing the existing skip list structure (**lst**) and a standard integer variable (**key**).*
- *Because these take a small, constant amount of memory (typically 8 bytes for **lst** pointer variable in 64 – bit architecture) and 4 bytes for **key** integer variable.*
*that never scales with the number of elements (**n**),*
*the parametric input space is strictly **$O(1)$** .*

B. Full Data Holistic Input Space: $O(n)$

- *The function operates on the entire pre – existing Skip List data structure.*
- *Because the expected maximum number of pointers per node remains a small mathematical constant, the total memory footprint to store n nodes scales directly and linearly with the number of elements.*
- *Therefore, the full data holistic input space is $O(n)$.*

2. Auxiliary Space Complexity: $O(1)$

Here is where the Search module becomes drastically simpler to write about than the Insertion module!

- ***The Insertion Contrast:*** *In insertion analysis, the primary consumer of auxiliary memory was the update array, which temporarily stored the memory addresses of predecessor nodes. Because that array had to scale with the maximum height of the list, it introduced a theoretical auxiliary space of $O(\log_2 n)$ (even though it was practically capped at $O(1)$).*

***[This we have already discussed in Insertion Element's
Space Complexity]***

- ***The Search Reality:*** *The `searchElement` module does not use an update array. It does not need to remember its path to rewire pointers later.*

- ***The Conclusion:*** *To execute a search, the algorithm only requires a single temporary pointer (**current**) and a loop counter variable (**i**). Because it allocates zero arrays, makes zero recursive calls, and requests no extra memory, the auxiliary space complexity for searching is an absolute, strict **O(1)** in both theoretical and practical scenarios.*

C) Algorithm

Algorithm: SkipListSearch(LST, KEY)

1. [Initialize Pointer]

SET CURRENT := HEADER[LST].

2. [Traverse List Vertically and Horizontally]

Repeat for $i = \text{LEVEL}[LST]$ down to 0:

Repeat while $\text{FORWARD}[i][\text{CURRENT}] \neq \text{NULL AND INFO}[\text{FORWARD}[i][\text{CURRENT}]] < \text{KEY}$:

SET CURRENT := FORWARD[i][CURRENT].

[End of while Loop]

[End of for Loop]

3. [Step into Exact Match]

SET CURRENT := FORWARD[0][CURRENT].

4. [Verify Key Existence]

If CURRENT $\neq$ NULL AND INFO[CURRENT] = KEY, Then:

Write: "Found key ", KEY, " in the Skip List."

Else:

Write: "Key ", KEY, " not found."

[End of If Structure]

5. [Exit]

END OF ALGORITHM.
